

RAPPORT DE TP

Kubernetes & Kubeflow

Du conteneur isolé au cluster d'orchestration MLOps

Module : Cloud & Infrastructure — Séance 2
1ère année Cybersécurité — 2025/2026

1. Introduction

Cette séance fait suite à la séance Docker, où nous avons appris à créer des images et des conteneurs. L'objectif ici était de comprendre pourquoi Docker seul ne suffit pas à grande échelle, de découvrir Kubernetes (architecture, objets essentiels, commandes kubectl), puis de préparer notre cluster de laboratoire et de démarrer réellement l'installation de Kubeflow, la plateforme MLOps qui s'appuie sur ce cluster.

L'infrastructure de laboratoire utilisée comprend trois VMs Ubuntu 22.04 déployées sur nos deux hôtes ESXi : k8s-master (control plane, 4 vCPU, 8 Go RAM, 100 Go disque), k8s-worker1 et k8s-worker2 (workers, chacun 6 vCPU, 16 Go RAM, 200 Go disque). Kubernetes (via kubeadm) était déjà installé sur les trois VMs, et kubectl configuré sur le master.

2. TP 1 — Vérification de l'infrastructure de labo

Avant toute manipulation Kubernetes, j'ai vérifié que les trois VMs disposaient bien des ressources et de la connectivité nécessaires.

```
$ free -h
$ nproc
$ df -h /
```

Sur le master, environ 8 Go de RAM étaient disponibles, et 16 Go sur chacun des deux workers, avec le nombre de vCPU attendu (4 sur le master, 6 sur chaque worker). L'espace disque disponible sur les workers était bien supérieur aux 80 Go recommandés pour Kubeflow.

```
$ ping -c 3 k8s-worker1
$ ping -c 3 k8s-worker2
$ ping -c 3 google.com
```

Depuis le master, les deux workers répondaient au ping, et la résolution DNS externe vers google.com fonctionnait également — condition indispensable pour que Kubeflow puisse télécharger ses images Docker.

```
$ sudo systemctl status containerd
```

Sur chaque nœud, containerd était bien actif (active (running)), confirmant que le runtime de conteneurs fonctionnait correctement avant de poursuivre.

3. TP 2 — Vérification du cluster Kubernetes

```
$ kubectl get nodes -o wide
```

Les trois nœuds (k8s-master, k8s-worker1, k8s-worker2) étaient en statut Ready, avec la même version de Kubernetes (v1.28.4) sur les trois. k8s-master affichait le rôle control-plane, les deux workers n'avaient pas de rôle explicite, ce qui est normal.

```
$ kubectl get pods -n kube-system
```

Tous les Pods du namespace kube-system étaient en Running : coredns (DNS interne, 2 instances), etcd-k8s-master, kube-apiserver-k8s-master, kube-controller-manager-k8s-master, kube-scheduler-k8s-master (les composants du control plane), kube-proxy (un par nœud, pour le routage réseau) et calico-node (plugin réseau, gestion du réseau inter-pods).

```
$ kubectl get ns
```

À ce stade, seuls les quatre namespaces par défaut étaient présents : default, kube-node-lease, kube-public et kube-system.

```
$ kubectl cluster-info
```

La commande a confirmé que le control plane Kubernetes tournait bien sur <https://192.168.1.10:6443>, avec CoreDNS accessible via le proxy de l'API. Le cluster était donc sain et prêt pour la suite.

4. TP 3 — Rappel rapide : Pod / Deployment / Service

Avant de passer à Kubeflow, j'ai réalisé un exercice court pour vérifier la maîtrise du cycle de base Deployment → Pods → Service, en créant le fichier test-nginx.yaml :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: test-nginx
  namespace: default
spec:
  replicas: 2
  selector:
    matchLabels:
      app: test-nginx
  template:
    metadata:
      labels:
        app: test-nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: test-nginx-svc
  namespace: default
spec:
  type: NodePort
  selector:
    app: test-nginx
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080

$ kubectl apply -f test-nginx.yaml
deployment.apps/test-nginx created
service/test-nginx-svc created
$ kubectl get pods -o wide
```

Les deux répliques du Deployment ont été automatiquement réparties par le scheduler : une sur k8s-worker1, l'autre sur k8s-worker2, chacune avec sa propre IP interne (10.244.x.x) attribuée par le plugin réseau.

```
$ kubectl get svc test-nginx-svc
$ curl http://192.168.1.11:30080
```

Le Service de type NodePort exposait bien l'application sur le port 30080 de n'importe quel nœud du cluster, et le curl a renvoyé la page d'accueil nginx. Cet exercice a confirmé le mécanisme de base : un Deployment crée des Pods répartis par le scheduler, un Service les expose de façon stable — exactement ce que fait Kubeflow à plus grande échelle.

```
$ kubectl delete -f test-nginx.yaml
```

Les ressources de test ont été supprimées avant de passer à la suite.

5. TP 4 — Préparation de l'environnement pour Kubeflow

Nous avons utilisé la méthode des manifests officiels Kubeflow avec kustomize, qui a l'avantage d'être transparente sur ce qui est réellement déployé.

5.1 Vérification et installation des outils

```
$ kubectl version --short
$ kubectl kustomize --help | head -3
$ curl -s "https://raw.githubusercontent.com/kubernetes-sigs/kustomize/master/hack/install_kustomize.sh" | bash
$ sudo mv kustomize /usr/local/bin/
$ kustomize version
$ sudo apt update && sudo apt install -y git
```

kustomize est intégré à kubectl depuis la version 1.14, mais une version standalone récente a été installée comme recommandé pour Kubeflow. git a également été installé pour récupérer les manifests.

5.2 Provisionnement du stockage persistant

```
$ kubectl apply -f https://raw.githubusercontent.com/rancher/local-path-provisioner/v0.0.26/deploy/local-path-storage.yaml
$ kubectl patch storageclass local-path -p '{"metadata": {"annotations": {"storageclass.kubernetes.io/is-default-class": "true"}}}'
$ kubectl get storageclass
```

local-path-provisioner de Rancher a été installé et défini comme StorageClass par défaut (local-path (default), provisioner rancher.io/local-path). Ce provisionneur stocke les données directement sur le disque du worker où tourne le Pod : ce n'est pas adapté à la production (pas de réplication, données perdues si le worker est perdu), mais c'est suffisant pour notre labo.

5.3 Récupération des manifests Kubeflow

```
$ cd ~
$ git clone https://github.com/kubeflow/manifests.git
$ cd manifests
$ git checkout v1.9-branch
$ ls -la
```

Nous avons utilisé la branche stable v1.9-branch plutôt que main pour éviter les régressions. La structure du dépôt contient notamment common/ (Istio, cert-manager, OIDC, namespaces partagés), apps/ (dashboard, notebooks, pipelines...) et contrib/ (extensions optionnelles).

À l'issue de ce TP, les outils étaient installés, le stockage persistant configuré, et les manifests Kubeflow récupérés : l'environnement était prêt pour l'installation proprement dite.

6. TP 5 — Installation de Kubeflow

C'est le cœur du TP : déployer l'intégralité de la plateforme Kubeflow en une seule commande à partir de ~/manifests. Cela inclut Istio (service mesh), cert-manager (certificats TLS), Dex et l'OIDC AuthService (authentification), le namespace kubeflow, le Central Dashboard, le Notebooks Controller, Kubeflow Pipelines, KServe, Katib, le Training Operator, les Profils/KFAM et la Volumes Web App.

```
$ cd ~/manifests
$ while ! kustomize build example | kubectl apply -f -; do
  echo "Retrying to apply resources..."
  sleep 10
done
```

La boucle while était nécessaire car l'installation crée des ressources dans un ordre précis : certaines dépendent de CRDs (Custom Resource Definitions) qui doivent exister avant d'être référencées. Plusieurs itérations avec des erreurs temporaires sont normales avant que toutes les dépendances soient satisfaites.

Pendant l'installation, un second terminal SSH a été ouvert pour surveiller la progression :

```
$ watch -n 5 "kubectl get pods -A | grep -v Running | grep -v Completed"
```

Au début, de nombreux Pods étaient en ContainerCreating ou Init, puis sont progressivement passés en Running. L'installation a suivi les quatre phases attendues : création des CRDs et namespaces (premières secondes), démarrage des composants d'infrastructure Istio/cert-manager/Dex (1 à 5 minutes), démarrage des composants applicatifs Kubeflow (5 à 15 minutes), puis stabilisation de l'ensemble des Pods en Running (15 à 20 minutes). Au total, l'installation a pris environ 20 minutes.

7. TP 6 — Observation du déploiement

7.1 Namespaces créés par Kubeflow

```
$ kubectl get ns
```

Par rapport aux 4 namespaces par défaut initiaux, plusieurs nouveaux namespaces sont apparus : auth (Dex et OIDC AuthService, authentification), cert-manager (gestion automatique des certificats TLS), istio-system (service mesh Istio, gère le trafic réseau et la sécurité), kubeflow (composants principaux : dashboard, contrôleurs, pipelines), knative-serving et knative-eventing (serverless pour KServe), et kubeflow-user-example (espace de travail du profil utilisateur par défaut).

7.2 Pods par namespace

```
$ kubectl get pods -n kubeflow
$ kubectl get pods -n istio-system
$ kubectl get pods -n auth
$ kubectl get pods -n cert-manager
```

Dans le namespace kubeflow, on retrouve notamment centraldashboard (2/2 Running), jupyter-web-app-deployment (2/2), katib-controller, katib-db-manager, katib-mysql, kserve-controller-manager (2/2), kubeflow-pipelines-profile-controller, ml-pipeline et ml-pipeline-ui (2/2), notebook-controller-deployment (2/2), profiles-deployment (3/3) et training-operator. Les namespaces istio-system, auth et cert-manager montraient également tous leurs Pods en Running.

7.3 Recherche des Pods problématiques

```
$ kubectl get pods -A --field-selector=status.phase!=Running,status.phase!=Succeeded
```

Cette commande n'a renvoyé aucun résultat : tous les Pods étaient soit en Running, soit en Completed (pour les jobs d'initialisation déjà terminés), confirmant que l'installation s'était bien déroulée.

8. TP 7 — Vérification des composants essentiels

```
$ kubectl get pods -n kubeflow -l app=centraldashboard
$ kubectl get svc -n kubeflow centraldashboard
$ kubectl get pods -n kubeflow -l app=ml-pipeline
$ kubectl get svc -n kubeflow ml-pipeline
$ kubectl get svc -n kubeflow ml-pipeline-ui
$ kubectl get pods -n kubeflow -l app=notebook-controller
$ kubectl get pods -n istio-system
$ kubectl get svc -n istio-system istio-ingressgateway
$ kubectl get deployments -n kubeflow
```

Tous les composants vérifiés étaient opérationnels : Central Dashboard (Running, 2/2), ML Pipeline et Pipeline UI (Running, 2/2), Notebook Controller (Running, 2/2), Istio Ingress Gateway (Running, avec un Service ClusterIP exposant les ports 80 et 443), Dex (namespace auth, Running) et cert-manager (Running). Dans `kubectl get deployments -n kubeflow`, la colonne READY affichait bien X/X pour chaque Deployment, sans aucun 0/X.

Composant	Namespace	Statut observé
Central Dashboard	kubeflow	Running (2/2)
ML Pipeline	kubeflow	Running (2/2)
Pipeline UI	kubeflow	Running (2/2)
Notebook Controller	kubeflow	Running (2/2)
Istio Ingress Gateway	istio-system	Running
Dex	auth	Running
cert-manager	cert-manager	Running

9. TP 8 — Accès initial à Kubeflow

Le service Istio Ingress Gateway n'ayant pas d'IP externe dans notre labo (pas de load balancer), l'accès au dashboard s'est fait via port-forwarding kubectl :

```
$ kubectl port-forward svc/istio-ingressgateway -n istio-system 8080:80 --address 0.0.0.0
```

L'option `--address 0.0.0.0` a permis d'accéder au port-forward depuis un autre poste du réseau, pas seulement en local sur le master. Le navigateur a ensuite été ouvert sur `http://<IP-du-master>:8080`.

La page de connexion Dex est apparue, et la connexion a été effectuée avec les identifiants par défaut du labo (`user@example.com / 12341234`). Une fois connecté, le Central Dashboard de

Kubeflow s'est affiché correctement, avec son menu latéral (Notebooks, Pipelines, Experiments, Runs, Volumes), le sélecteur de namespace en haut de la page, et les sections Notebooks/Pipelines/Volumes accessibles.

L'accès au dashboard confirme que l'installation de Kubeflow est réussie : nous disposons d'un cluster Kubernetes avec une plateforme MLOps complète et opérationnelle, prête à être exploitée dans les prochaines séances.

10. TP 9 — Comprendre ce qui a été installé

```
$ kubectl get pods -A | wc -l
$ kubectl get pods -A --no-headers | awk '{print $1}' | sort | uniq -c | sort -rn
```

Le cluster comptait plusieurs dizaines de Pods au total, répartis principalement ainsi : 18 dans kubeflow, 8 dans istio-system, 5 dans cert-manager, 5 dans kube-system, 3 dans auth, 3 dans knative-serving, 2 dans knative-eventing, et 1 dans kubeflow-user-example. Le namespace kubeflow concentre donc le plus grand nombre de composants.

Cette répartition correspond aux différentes couches de la plateforme : l'infrastructure (Istio, cert-manager) assure un réseau sécurisé et la gestion des certificats de manière transparente ; l'authentification (Dex, OIDC AuthService) intervient à chaque connexion au dashboard ; l'interface (Central Dashboard) constitue le point d'entrée permanent ; les notebooks (Notebook Controller, Jupyter Web App) serviront dès la séance 3 pour l'exploration et le prototypage ; les pipelines (ML Pipeline, Pipeline UI, Argo) serviront aux séances 4-5 pour l'automatisation MLOps ; le serving (KServe, Knative) et l'entraînement (Training Operator, Katib) seront utilisés dans les séances futures ; et les Profils/KFAM assurent dès maintenant l'isolation multi-utilisateurs, visible dans le dashboard.

L'essentiel à retenir : Kubeflow n'est pas une application unique mais une plateforme composée de plusieurs briques, chacune étant un ensemble de Pods, Services, Deployments et ConfigMaps — les mêmes objets Kubernetes vus en théorie. Istio et l'authentification forment la couche invisible qui fait fonctionner l'ensemble.

11. TP 10 — Conservation de l'environnement

11.1 Documentation de l'état du cluster

```
$ kubectl get pods -A -o wide > ~/etat-cluster-seance2.txt
$ kubectl get svc -A >> ~/etat-cluster-seance2.txt
$ kubectl get ns >> ~/etat-cluster-seance2.txt
$ echo "Date: $(date)" >> ~/etat-cluster-seance2.txt
```

Un snapshot de l'état du cluster (Pods, Services, namespaces, date) a été enregistré dans un fichier texte, qui servira de référence pour la prochaine séance.

11.2 Sauvegarde des VMs

Un snapshot VMware a été pris pour chacune des trois VMs via vCenter (clic droit sur la VM → Snapshots → Prendre un snapshot), nommé Séance2-Kubeflow-installé, avec inclusion de la mémoire pour permettre une restauration sans redémarrage complet.

11.3 Ordre d'arrêt et de redémarrage

Pour un arrêt propre entre les séances, l'ordre recommandé est : arrêter d'abord k8s-worker2, puis k8s-worker1, et enfin k8s-master en dernier. Au redémarrage, l'ordre inverse est appliqué : master en premier, puis les workers. Il faut prévoir 5 à 10 minutes après un redémarrage complet avant que tous les Pods reviennent en Running, le temps que les health checks et readiness probes se stabilisent.

12. Méthode de lecture d'une installation Kubernetes

Pour analyser n'importe quelle plateforme déployée sur Kubernetes (pas seulement Kubeflow), la méthode suivie a été la suivante, en quatre étapes :

1. Namespaces (`kubectl get ns`) : donne une carte de haut niveau, chaque namespace correspondant à un domaine fonctionnel.
2. Pods (`kubectl get pods -n <ns>`) : montre les composants actifs ; le nom du Pod contient souvent celui du composant, et READY/STATUS indiquent sa santé.
3. Services (`kubectl get svc -n <ns>`) : montre les points d'accès, le type (ClusterIP, NodePort, LoadBalancer) indiquant si le service est interne ou externe.
4. PersistentVolumeClaims (`kubectl get pvc -n <ns>`) : montre quels composants utilisent du stockage persistant et quelle quantité.

À cela s'ajoute la compréhension du rôle d'Istio (routage réseau via VirtualServices et Gateways) et de Dex (authentification), deux composants invisibles au quotidien mais essentiels au fonctionnement global.

13. Réponses à l'évaluation

13.1 Questions de vérification

5. Docker est un outil de conteneurisation : il permet d'empaqueter et d'exécuter une application de façon isolée et reproductible, sur une seule machine. Kubernetes est un outil d'orchestration : il automatise la gestion de nombreux conteneurs à grande échelle (redémarrage automatique, scaling, répartition multi-nœuds, mise à jour sans interruption, service discovery). On a besoin des deux car ils sont complémentaires : Kubernetes utilise Docker (ou un autre runtime comme containerd) pour exécuter les conteneurs, mais ne remplace pas Docker pour la construction des images.
6. Les 4 composants principaux du Control Plane sont : kube-apiserver, point d'entrée unique pour toutes les interactions avec le cluster (chaque commande `kubectl` y passe) ; etcd, la base de données clé-valeur qui stocke l'état complet du cluster ; kube-scheduler, qui décide sur quel nœud worker placer chaque Pod selon les ressources disponibles ; et controller-manager, qui exécute les boucles de contrôle vérifiant en permanence que l'état réel correspond à l'état déclaré.
7. Un Pod est l'unité d'exécution la plus petite dans Kubernetes : il contient un ou plusieurs conteneurs partageant le même espace réseau et le même stockage. Il diffère d'un simple conteneur Docker car Kubernetes ne manipule jamais directement un conteneur isolé : il l'encapsule toujours dans un Pod, qui ajoute une couche d'abstraction (réseau partagé, cycle de vie géré, possibilité de plusieurs conteneurs co-localisés comme un conteneur principal et un sidecar).

8. Un Service expose un ensemble de Pods sous une adresse réseau stable (IP et nom DNS). Les Pods ne suffisent pas pour accéder à une application car ils sont éphémères : ils peuvent être recréés à tout moment par le scheduler (sur un autre nœud, avec une nouvelle IP). Le Service maintient un point d'accès permanent indépendant du cycle de vie des Pods individuels, et assure également la répartition de charge entre les répliques.
9. Kubeflow a besoin de Kubernetes car chaque notebook, chaque étape de pipeline, chaque composant d'interface s'exécute comme un Pod, géré via les primitives Kubernetes (Deployments, Services, ConfigMaps, PersistentVolumes, namespaces). Deux exemples concrets observés dans notre installation : le Central Dashboard est un Deployment exposé via un Service dans le namespace kubeflow, et le Notebook Controller crée dynamiquement un Pod (avec un PersistentVolume monté) chaque fois qu'un utilisateur lance un notebook Jupyter.
10. La commande `kubectl get pods -A` permet de voir tous les Pods du cluster, dans tous les namespaces. Le flag `-A` (all namespaces) est important car par défaut `kubectl` n'affiche que le namespace default : sans ce flag, on ne verrait ni les composants système (kube-system) ni les composants Kubeflow (kubeflow, istio-system, auth, etc.), ce qui donnerait une vision très incomplète de l'état du cluster.
11. Après l'installation de Kubeflow, 7 namespaces supplémentaires sont apparus par rapport aux 4 namespaces par défaut : auth, cert-manager, istio-system, knative-eventing, knative-serving, kubeflow et kubeflow-user-example. Trois exemples avec leur rôle : istio-system héberge le service mesh Istio qui gère le trafic réseau, la sécurité et l'observabilité ; kubeflow contient les composants principaux de la plateforme (dashboard, contrôleurs, pipelines) ; et auth héberge Dex et l'OIDC AuthService, responsables de l'authentification.
12. Dex est le système d'authentification et de gestion d'identité (OIDC) de Kubeflow. Son rôle est de présenter la page de connexion lors de l'accès au dashboard et de gérer les identités des utilisateurs (par défaut, le compte `user@example.com / 12341234` dans notre labo). En production, Dex serait configuré pour s'interfacer avec un vrai fournisseur d'identité (LDAP, OAuth2) plutôt que d'utiliser des identifiants par défaut.
13. Un Pod en état `ImagePullBackOff` signifie que le nœud n'a pas pu télécharger l'image Docker du conteneur. Les causes probables sont un problème de connectivité Internet ou de résolution DNS sur le nœud, un nom ou tag d'image incorrect, ou un registry inaccessible/nécessitant une authentification. Pour diagnostiquer, on utilise `kubectl describe pod <nom> -n <namespace>`, qui affiche dans la section Events le message d'erreur exact concernant le pull de l'image.
14. La méthode en 4 étapes pour lire une installation Kubernetes est : ① Namespaces (vue d'ensemble des domaines fonctionnels) → ② Pods (composants actifs et leur état de santé) → ③ Services (points d'accès internes ou externes) → ④ PersistentVolumeClaims (stockage utilisé par chaque composant). Cette séquence fonctionne pour n'importe quelle plateforme déployée sur Kubernetes, pas seulement Kubeflow.

13.2 Questions de réflexion

Question 1 — Limites de local-path-provisioner et alternative en production

local-path-provisioner stocke les données directement sur le disque local du worker où tourne le Pod, sans aucune réplication. Les limites principales sont les suivantes : si le worker tombe en panne ou est supprimé, toutes les données qui y étaient stockées sont définitivement perdues ;

un Pod utilisant un volume créé par ce provisionneur reste lié au nœud où le volume a été créé (le mode WaitForFirstConsumer empêche en pratique de migrer facilement le Pod vers un autre nœud) ; et il n'y a aucune fonctionnalité de sauvegarde, de snapshot ou de haute disponibilité intégrée. En production, il faudrait utiliser une solution de stockage distribué et répliqué, comme Ceph, Longhorn, ou un stockage réseau partagé (NFS, iSCSI), qui garantit que les données restent accessibles même si le nœud d'origine est perdu, et permet de déplacer librement les Pods entre les nœuds du cluster.

Question 2 — Pourquoi Kubeflow installe Istio, et impact si Istio ne fonctionne pas

Kubeflow installe Istio car ce service mesh fournit la couche de routage, de sécurité et d'observabilité du trafic entre tous les composants de la plateforme : c'est lui qui gère l'Istio Ingress Gateway, point d'entrée unique par lequel transite tout le trafic vers le dashboard et les autres interfaces, et qui permet d'appliquer des règles de sécurité (chiffrement du trafic interne, contrôle d'accès) de manière transversale sans modifier chaque composant individuellement. Istio fournit également les VirtualServices et Gateways qui dirigent les requêtes vers le bon Service selon l'URL demandée. Si Istio ne fonctionnait pas, l'impact serait majeur : l'Ingress Gateway ne pourrait plus router le trafic entrant, ce qui rendrait le Central Dashboard et l'ensemble des interfaces web de Kubeflow inaccessibles, même si les Pods applicatifs (dashboard, pipelines, notebooks) continuaient eux-mêmes de tourner normalement en arrière-plan. Istio est donc un composant d'infrastructure critique, invisible au quotidien mais indispensable.

Question 3 — Pods READY 2/2 : rôle du second conteneur

Dans un Pod affichant READY 2/2, le second conteneur est généralement un sidecar Istio (istio-proxy, basé sur Envoy), injecté automatiquement par Istio dans chaque Pod du mesh. Ce conteneur sidecar intercepte tout le trafic réseau entrant et sortant du conteneur applicatif principal : il applique les règles de routage, de sécurité (TLS mutuel entre services) et collecte les métriques d'observabilité, sans que l'application elle-même ait à en avoir connaissance. C'est pour cette raison que de nombreux Pods dans le namespace kubeflow affichent 2/2 (le conteneur applicatif + le sidecar istio-proxy), alors que des Pods qui ne font pas partie du mesh Istio (par exemple certains composants de cert-manager) restent en 1/1.

14. Conclusion

Cette séance nous a fait passer de la manipulation de conteneurs isolés à un cluster Kubernetes complet faisant tourner une plateforme MLOps. Nous avons vérifié l'infrastructure de laboratoire (ressources, réseau, runtime), validé la santé du cluster Kubernetes à trois nœuds (control plane + 2 workers), réalisé un exercice de rappel Deployment/Service avec nginx, puis préparé l'environnement (kustomize, git, stockage persistant local-path) avant de lancer l'installation complète de Kubeflow via les manifests officiels.

L'installation, qui a duré environ 20 minutes, a déployé plusieurs dizaines de Pods répartis sur 7 nouveaux namespaces (kubeflow, istio-system, auth, cert-manager, knative-serving, knative-eventing, kubeflow-user-example). Nous avons vérifié les composants essentiels (Central Dashboard, ML Pipeline, Notebook Controller, Istio Ingress Gateway, Dex, cert-manager), tous en Running, puis accédé avec succès au Central Dashboard via port-forwarding et les identifiants par défaut du labo.

Au-delà de l'installation elle-même, cette séance nous a donné une méthode générale pour lire toute installation sur Kubernetes (namespaces → pods → services → PVC), et a clarifié l'articulation entre Docker (conteneurisation), Kubernetes (orchestration) et Kubeflow (plateforme MLOps construite sur ces deux couches). Le cluster a été documenté et sauvegardé (snapshots VMware) en vue de la séance suivante, consacrée à l'utilisation concrète des notebooks Jupyter de Kubeflow.